

Linguistic State Machines: Modeling Non-Playable Character Behavior through Natural Language Descriptors

Chandra Reka Ramachandiran

Dept. of Software Engineering

Xia Men University Malaysia

Selangor, Malaysia

chandrareka.ramachandiran@xmu.edu.my

William Hardjoko

Dept. of Computer Science and Technology

Xia Men University Malaysia

Selangor, Malaysia

william.hardjoko07@gmail.com

Abstract— Non-Playable Characters (NPCs) are an essential game element that, in many games, interacts the most with the player and contributes substantially to the player's immersion. However, traditional models demand each behavior be explicitly defined. As a result, modeling and adjusting NPC behaviors requires considerable time and effort. Furthermore, player interactions with traditional NPCs are often scripted and provide limited interactions. This work demonstrates the viability of employing Large Language Models in behavioral modeling. Our approach enables designers to guide behaviors rather than describing precise instructions. The system also allows players to interact with the NPC in free-form natural language. To evaluate the efficacy of our method, we developed a simple game based on this premise. We investigate the strengths and limitations of our approach from a development and gameplay experience standpoint. Additionally, we are interested in assessing the role-play consistency of our framework. We present our analysis and findings while also suggesting directions for further study.

Keywords— Non-Playable Character, Large Language Model, Behavioral Modeling, Game Development

I. INTRODUCTION

The development of Large Language Models (LLMs) has drawn much attention to Natural Language Generation (NLG). Aside from text generation, these models are proficient in, among other things, analogical reasoning [1], following instructions [2], data analysis [3], and role-play [4], [5]. In this work, we aim to investigate the potential application of Large Language Models for modeling Non-Playable Character (NPC) behaviors in video games. Existing behavioral modeling constructs follow the do-as-I-say paradigm. They focus on providing a high-level view, robust control, and easy-to-understand primitives. Each behavior in these systems requires explicit elaboration on the actions and execution conditions, such as locations, character states, etc. Because of that, they often are rigid and struggle to scale with more complex and rich characters [6], [7].

Alternatively, Large Language Models open the avenue for a *guide when needed* paradigm shift. These models are trained with the conversational style and knowledge of many characters [4], [5], implying that they could predict and portray the actions of numerous characters in various settings. Hence, given a character description, this system allows designers to quickly set up, interact, and guide the NPC's behavior through natural language. Rather than

describing a low-level system, this approach is akin to instructing an actor how to behave on a set. This methodology would offset the need for explicit behavioral structure, saving time and effort in development.

This study investigates the potential application of Transformer-based models like GPT-4 [8] for guide-as-needed behavioral modeling in video games. To this end, we developed our framework on the premise that any behavior model is simply a collection of context, trigger, and action. In this context, the term "context" refers to attributes that may affect a character's decision process, such as:

1. **Character Profile.** A description containing the character traits, goals, beliefs, etc.
2. **In-game States/Surrounding.** A collection of in-game data that describes the current character's condition in the environment.
3. **World Settings.** A brief description of the in-game fantasy settings.
4. **Character Knowledge.** A set of information containing the character's observations up until now, such as events, social interactions, schedules, current goals, etc.

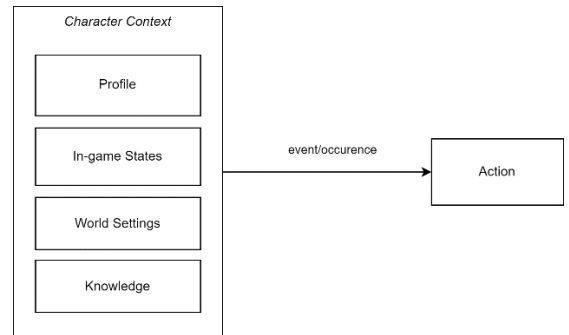


Fig. 1 Simplified Behavior Design Model

Our approach employs an LLM to create a sequence of actions based on the context and stimuli, which would trigger the in-game NPC to act. Because LLMs are a superposition of characters [5], we posit that LLMs could correctly infer character behaviors based on the settings, conditions, and character traits.

Assessing the efficacy of our framework poses a significant challenge. There are no efficient and reproducible procedures for a behavior modeling construct. Evaluation typically involves years of development

experience and user interactions with the NPCs. For the interest of this paper, we aim to assess both factors as follows:

- (1) **Development experience.** We develop a simple game with Linguistic State machine-based NPCs as its core mechanic. This process would help us gather insights about the technical strengths and challenges.
- (2) **Role-play consistency.** While maintaining a role-play identity is not an issue for other behavior modeling frameworks, LLMs are known to hallucinate and may break characters. Hence, we designed a metric based on related works of literature [5]. After interacting with our demo game, we would ask participants to fill out a survey form that measures these metrics. Although indirect, the perceived role-play consistency would help us infer the average user experience interacting with the system.

II. RELATED WORKS

Classical Behavioral Model. Early behavioral models often focus on simple high-level constructs that could generalize to low-level code. A widespread example is the Finite State Machine (FSM) [9], [10]. In FSM, states and transitions describe behaviors. Behaviors are executed depending on the state and the event the machine receives. Another popular contender is the Behavior Tree [7]. In this framework, behaviors are organized in a tree-like format. However, these frameworks often face scaling issues as the project grows and behavioral components become more complex.

For example, the number of states and transitions in the Finite State Machine grows exponentially increasingly complex behaviors [11]. This exponential growth clutters the model [7], making it increasingly hard to understand, maintain, and implement [6]. Hierarchical Finite State Machine, a variant of FSM, mitigates this issue by grouping reoccurring states and transitions into higher-level states [12].

Although to a lesser degree, Behavior Tree faces the same scaling issues. Large Behavior Tree generally consumes plenty of memory [13], which is problematic with complex or persistent behaviors. Additionally, Behavior Trees are typically slower than the alternative, the Finite State Machine [12]. However, since there is no standard implementation of the Behavior Tree, these issues are inconsistent between pieces of literature [7].

Machine-Learning Aided Model. Given the robust nature of traditional models, they demand a significant effort to alter to meet specific design requirements [14]. Additionally, gameplay experiences are sometimes challenging to describe in behavior model primitives. Systems under this category incorporate Machine-Learning algorithms to facilitate the creation of behavior constructs, commonly Behavior Trees. One use case for this method is programming by demonstration [15], [16]. These systems observe a live demonstration and attempt to construct a Behavior Tree to model said observations. Another use case is for aiding the creation of emergent gameplay experiences. These experiences refer to a gameplay experience that emerges from a composition of sub-

behaviors rather than a specific behavior, such as difficulty levels [14]. However, techniques in this family often produce exponentially more complex models than the initial construct [16]. Hence, the result may harm the development rather than aid [17].

Large Language Model Driven Model. Despite the impressive capabilities of Large Language Models [1], [2] and their remarkable ability to learn from examples [18], few literatures utilize them for modeling NPC behavior. Research in this area often focuses on LLMs' application as a dialog generator [19], [20]. Meanwhile, research utilizing LLM for controlling in-game characters tends to overlook the role-playing aspect of these models [21]. One area of concern for employing LLMs in behavioral prediction appears to be performance [19]. However, we argue that the strength of employing Large Language Models comes from their versatility and not necessarily performance-wise. We believe newer architectures will offset the performance compromise in time [22].

III. METHODOLOGY

Our modeling approach consists of *Function Markup Language* (FML) and *Linguistic State Machine* (LSM). Linguistic State Machine is our proposed approach for modeling Non-Playable Character behaviors, while FML serves as a hallucination reduction mechanism. We present the general form and a variant we use in our demo game for each component. We hope the two serve as a starting point and provide insight into the additional systems required in a production environment.

A. Linguistic State Machine

In a Linguistic State Machine (LSM), a *state* represents a scenario the character is in. A state should contain:

- (1) **Memory.** A list of facts or general knowledge the character is aware of. This list may contain a log of the actions it has taken until now.
- (2) **States.** A list of in-game data that represents the NPC's condition. This list includes internal states, such as emotion and goals, and the character's observation of its surroundings, like nearby items and current location.
- (3) **Actions.** A list of valid actions the NPC could take in the current condition. In implementation, this list contains the name available functions with natural-sounding names.

Meanwhile, a "transition" refers to a change of state from the state I to $I + 1$, triggered by an NPC observable in-game event. When transitioning between states, we encode the current state into a prompt and embed the triggering events. Afterward, we prompt the Large Language Models L to generate function calls representing the character's subsequent actions. The game then evaluates the functions to apply the actions to the in-game NPC. Afterward, the value of the states may or may not change, completing the transition from I to $I + 1$. Fig. 2 concisely describes this model.

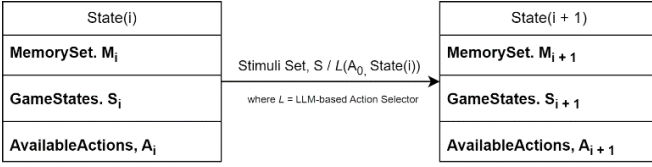


Fig. 2 General Form of Linguistic State Machine

During development, we observe that players might interact with the system multiple times during natural language generation, effectively interrupting the NPC. This circumstance would also naturally occur in modern games and should be handled to prevent outdated responses. However, the base formulation does not accommodate interruptions. Hence, we devise a simple interruption model using the Finite State Machine, as illustrated in Fig. 3. This model handles interruption by assuming the current events are stale and requests the language generation service to stop. Afterward, the system pushes the current events into history and restarts the transition process to handle the interrupting events

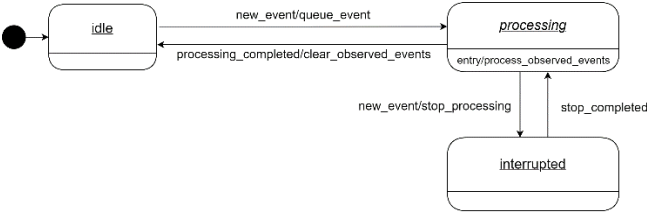


Fig. 3 Finite State Machine Model of Interruptible Linguistic State Machine

```
// Name: Lisa
// Personality: Generous, kind, loving, critical
// In-game State:
// Inventory: ['dirt', 'potion', 'glass']
// PeopleAroundMe: ['stranger', 'mom']
// Knowledge:
// (EMPTY)
// Recent Memories:
// (EMPTY)
// Event:
// - A stranger said: 'Can I have your potion, please?'
// Available function:
// - give(item_name :string, people_around_me: String)
// - chat(message: String, target_people: String)
```

Fig. 4 An Example Linguistic State Machine Prompt for A Kind Witch

```
give('potion', 'stranger')
// Justification: Lisa has a potion in her inventory
```

Fig. 5 Generated Function JavaScript Function Calls for the Kind Witch Prompt

B. Function Markup Language

One of the weaknesses of a Large Language Model is its tendency to hallucinate [23]. During development, our model sometimes creates a new JavaScript function instead of calling the provided ones or attempts to call a non-existent function. We posit that this deviation was caused by instructing the model to write in JavaScript. Hence, there is a non-zero probability that it attempts to use any JavaScript built-in features.

```
function handleEvent(event) {
  if (event.peopleAroundMe.includes('stranger')) {
    chat(event.message, event.peopleAroundMe)
  } else {
    give('potion', event.peopleAroundMe)
  }
}
```

Fig. 6 Invalid Model Output of Example Linguistic State Machine Prompt

This discovery poses a unique challenge. Every programming language naturally has a built-in functionality to support development. However, instructing the model to write in any programming language could trigger a similar case illustrated in Fig. 6. To accommodate this issue, we propose a barebone markup language for function calls, the Function Markup Language (FML). A general specification of FML are as follows:

- 1) FML programs only contain a single line and start with a function call.
- 2) Any line starting with a hashtag is a comment.
- 3) Any non-quoted continuous alphabetical characters are function names.
- 4) A function call begins with a function name followed by an opening bracket, a list of arguments separated by commas, and ends with a closing bracket.
- 5) An argument list could only contain primitive data types (string, integer, and float) and function calls with a return type (allows objects and classes).

As a standalone programming language, it is useless and impractical. However, its barebone nature forces a language model to generate function calls, removing the possibility of feature hallucinations. Moreover, an FML interpreter must link the function symbols to their implementations before evaluation. That means FML codes could run on many platforms and programming languages.

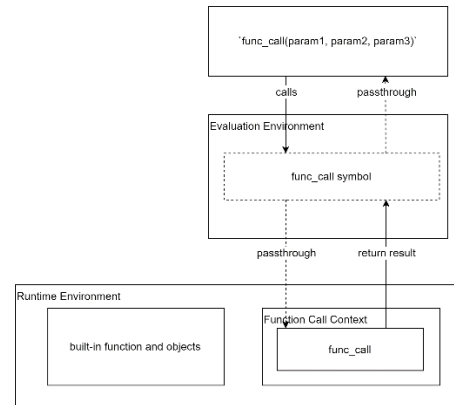


Fig. 7 Function Markup Language Interpreter Flow

In our implementation, we consider String the only valid primitive data type. A string could describe most NPC-related information, such as names, locations, and nearby objects. Hence, we define the FML grammar as illustrated in Fig. 8. Subsequent work could extend this grammar to include more language features, which may improve the conciseness of the generated function calls.

$$F \rightarrow f(A)$$

$$A \rightarrow s, A \mid F, A \mid s \mid F$$

where $s = \text{String}, f = \text{function name}$

Fig. 8 Function Markup Language Implementation Grammar

C. Model Selection

We select the *Kappa Phi-3* model for our project. Although this model categorically falls under the Small Language Model (SLM), its capabilities for learning from examples remain similar to Large Language Models [24]. Additionally, we acknowledge that response latency would matter to the gameplay experience. Hence, we decided to compromise on reasoning capabilities for a boost in performance.

D. Additional Classes

Our implementation of the Linguistic State Machine comprises of several classes, which are:

- (1) **State**. An abstract data class containing a value representing an NPC in-game state. It is also responsible for determining valid actions an NPC could take related to the state. For example, a “location” state might provide a “go to the location” action.
- (2) **Persona**. A class containing character descriptions. This class includes character knowledge, the world description, the character name and personal information, personality traits, etc. For more control, we also included a field named “Guidance” to allow designers to instruct and course-correct NPC behaviors.
- (3) **Agent**. The implementation of an interruptible Linguistic State Machine. This class is directly attached to the corresponding NPC.

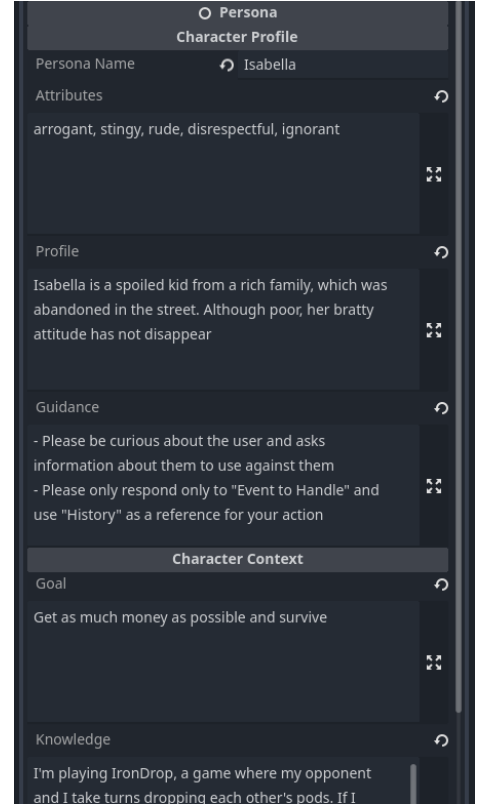


Fig. 9 Persona Class Filled with The Character Description for Isabella

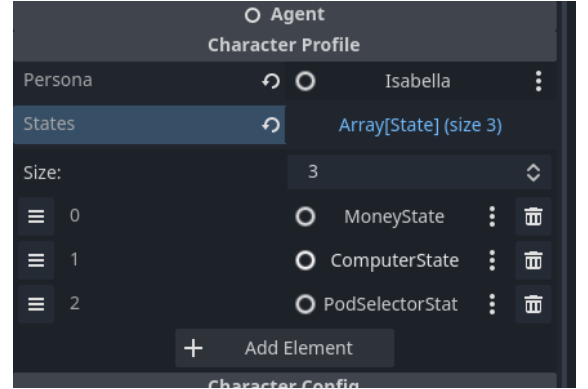


Fig. 10 Agent Class Implementation Running Isabella Persona

When the NPC observes an in-game event, it gathers all state values and valid actions from each state. Then, the PromptBuilder¹ class constructs a prompt based on the collected data and the events. Afterward, we prefix the prompt with a system instruction, FML syntax, and a few shot examples before sending it² to the language model service to generate the response FML code. Due to the properties of FML, the game eagerly evaluates the code each time the model generates a new line to reduce latency.

¹ We define a PromptBuilder class to easily switch between prompting structure. However, the components always have clear separations as illustrated in Fig. 4.

² We release the full source-code and prompt in this repository: <https://github.com/plasteek/IronDrop>

IV. EVALUATION

A. Metric Design

There is a lack of efficient evaluation methods for open-ended problems, such as role-play. This issue is exacerbated by utilizing LLMs as an action predictor rather than dialog generators. Hence, we designed a metric that focuses on measuring role consistency, drawing inspiration related literatures [5]. The attributes of our metrics are as follows:

Role Consistency. An NPC utilizing LLMs should behave according to their assigned role without deviation. Even when an unexpected or peculiar event occurs, the system should remain in character and continue depicting the character's response to said scenario. This metric also expects the system to be resilient to player meddling. A player might inappropriately interact with or speak nonsense. However, the system must retain the qualities of the character (conversational style, personality traits, etc.) while interacting with the player.

Knowledge Alignment. This metric assesses the system's knowledge regarding its role. Specifically, whether the system's knowledge aligns with the character or not. However, our system provides an explicit knowledge base for the model. Hence, this metric examines the system's tendency to provide information outside its knowledge base.

Request Rejection. A good NPC should ideally have a boundary for rejecting a player's request. They should typically reject a request if:

- 1) The request is outside the capabilities of the NPC.
- 2) The request misaligns with the character's role and descriptions. This misalignment may include a request that contradicts the character's personality traits, moral code, etc.

This factor differs from Role Consistency as it focuses on the rejection rather than the response.

B. User Study

After the user interacted with the demo video game, they are prompted to fill a survey form³, which draws heavy inspiration from the System Usability Scale [25]. The form is comprised of statements that contradict or align with the measured metric. The user would score each statement in the 5-point Likert scale, mapping from strongly disagree to strongly agree. For analysis, we map⁴ the 5-point Likert scales into a range of 1 to 5 (strongly disagree to strongly agree). Afterwards, we average the ratings of each metric across all the participant and compare it to the grading scale, illustrated in TABLE 1.

TABLE 1 RATING GRADING SCALE

Rating Range	Grade
1.0 – 2.9	Problematic
3.0 – 3.4	Average

³ We included the full survey form in the appendix.

3.5 – 4.2	Good
4.3 – 5.0	Excellent

V. EXPERIMENT

To measure the efficacy of our framework, we constructed a simple game that forces the player to interact with the NPC to progress. This demo draws inspiration from a Trust-based variant of the Centipede Game [26].

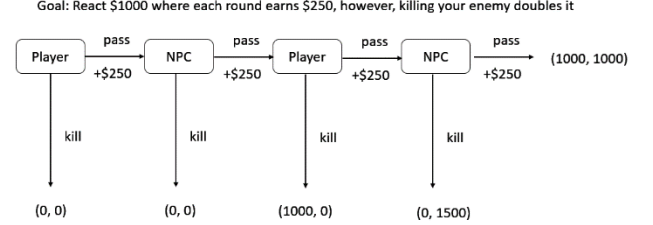


Fig. 11 High-level Overview of Trust-oriented Centipede Game

The gameplay loop roughly follows:

- 1) Each contestant (the player and the NPC) starts with zero credits.
- 2) In each turn, the deciding contestant could choose to either:
 - a. *Pass the turn.* This choice would pass the deciding role to the other contestant and add 250 credits to both.
 - b. *Kill.* This option would give the deciding contestant all the opponent's credits and end the game. Intuitively, this option doubles the credit of the deciding role.
- 3) If the credit of both players is under 1000, the game repeats step two.
- 4) If both player's credit is above 1000, the game ends with mutual victory.
- 5) If either player decides to kill, the resulting credit must be above 1000 to win. Otherwise, it would be considered a mutual loss.

Fig. 11 illustrates that, ideally, both players choose to cooperate and attain mutual benefit. However, Fig. 11 also highlights that in the 3rd and 4th turn, the player and the NPC could backstab the other party and gain higher rewards. Our twist comes with the uncertainty of "kill" and "pass." In our demo game, Iron Drop, the non-deciding contestant is placed inside a random pod, where the total may change from 2 to 4. The uncertainty encourages the player and the NPC to Fig. out the intent of either party, whether they are malicious or collaborative.

If the opposing party is hostile, the contestant may provide false information to prevent demise. On the other hand, if the opposing party is cooperative, both parties would benefit from being truthful to guarantee mutual benefit. This design allows the player to win regardless of the opposition's intent. Additionally, we add a scenario with two total pods, effectively creating a 50-50 situation

⁴ For contradicting statements, we inverse the resulting range by subtracting the number 6 with the user score to compute the user rating if the statement is not contradictory.

for both contestants. In this scenario, a false distrust or naivety may result in a game loss.



Fig. 12 Player Pod Selection in Iron Drop

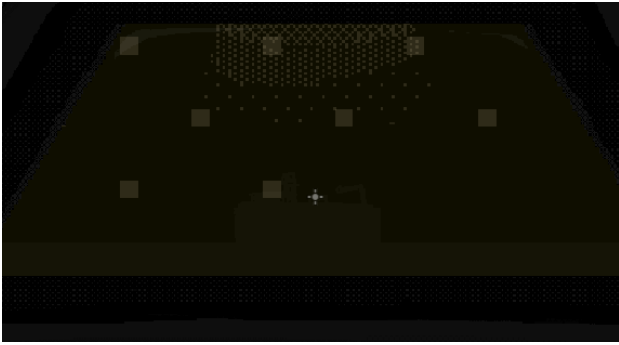


Fig. 13 Player Waiting for the NPC to Select Pod

Moreover, the demo features six unique adversaries. Three are actively malicious, while the others are potentially collaborative but have distrustful personality traits.

Before interacting with the game, we advise players to play the demo several times to encounter different personalities. Furthermore, we encourage players to alternate between greedy and collaborative strategies. Our goal is to acquire general data regarding the consistency and intelligence of the system. Lastly, we urge the user to reach out and provide feedback regarding their experience. With these guidelines, we hope players score each metric more objectively and offer valuable insights about the overall experience.

VI. RESULTS

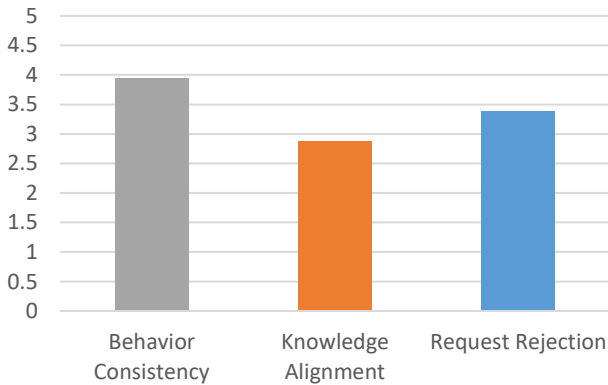


Fig. 14 Average Rating for Each Metric

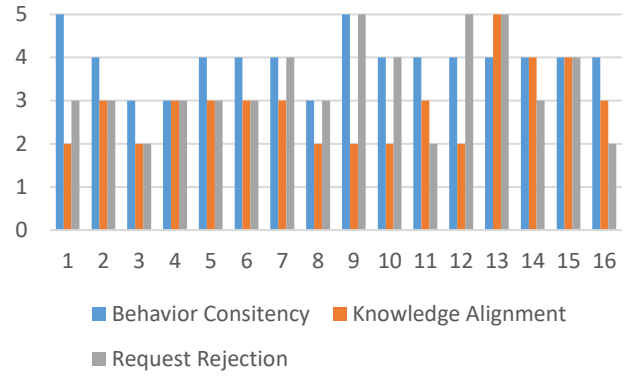


Fig. 15 Participant Scores for Each Metric

A. System Capabilities

After compiling our development experience and user feedback, we present the strengths of our framework as follows:

Rapid Persona Development. Despite featuring six unique personas, designing and adjusting their behaviors took only several hours. We found that the NPC behaves as intended with only minimum setup. Hence, their behaviors only require minor changes.

Ease of Guidance. In our implementation, we implemented an additional field named "*guidance*" to embed additional instructions for the system. The NPC appears to understand concrete instructions, such as "*Please greet the player before playing.*" and general guidance, like "*Please refrain from using curses and insults in conversations.*" Furthermore, these instructions appear to be followed consistently across testing.

Consistent Personality. We observed that the NPC maintains a consistent character depiction event and is resilient to player meddling. Attempts to break the system using non-sensical facts, dialogs, and insults cause the system to dismiss said attempts. Fig. 14 and Fig. 15 seem to support our observations and imply the system's good self-identity.

Capability Awareness. **Error! Reference source not found.** suggests that the NPC recognizes impossible or misaligned player requests. When rejecting a request, the NPC would improvise a reason that aligns with the setting and character.

Resilient Knowledge Base. Fig. 14 indicates that the system has knowledge hallucinations. However, we confirm that the NPC does not provide the player with information outside its knowledge base. We identify that the low score comes from the player's unawareness and the NPC's eagerness to utilize its knowledge base. These factors result in a misalignment between the player's expectations and the NPC's dialogs. We will discuss this further in the next section.

Multi-event Handling. The NPC is capable of recognizing and responding to multiple events. We

encountered this property during initial testing⁵ and did not use it in our implementation.

Maintains Code and Conversational Context.

Assuming the NPC logs its actions in memory and conversation is carried through a dedicated function, the NPC manages to generalize said function calls into a separate conversational context. As a result, it retains the ability to converse with the player. However, the players often perceive the conversations as disjointed. We identify the root of this issue lies in gameplay and not necessarily the system, which we will discuss in the subsequent sections.

B. Known Issues

A significant portion of the issues seems to stem from our prompting approach. Our approach solely relies on few-shot prompting and the model's capability for in-context learning [18]. We posit that better prompting techniques [27], [28] would resolve many of the system's issues. Our investigation into the limitations of our system yields the following:

The Knowledge Inclusion Problem. During user evaluation, one of our participants reported that a character actively compared him to "Robert." After a thorough inspection, we discovered that "Robert" was in the character's knowledge where: "Robert is my husband." This discovery poses an interesting question: "What should a character's knowledge contain?" This subject requires further studies and experimentation. Assuming the necessity of including similar pieces of information, knowledge distillation techniques [27] may substantially reduce these cases. Alternatively, embedding additional guidance would have similar effects. However, we do not recommend similarity-based Retrieval-Augmented Generation techniques [29]. Often, an NPC may require several seemingly unrelated pieces of information before acting. Assume a hypothetical knowledge base: "*Dad bought groceries*" and "*Groceries usually include milk, soap, and vegetables.*" If a character buys milk only if "*Dad*" has not, a similarity-based RAG would fail to retrieve the fact that groceries contain milk.

Non-Existent Function Calls. When responding to an event, the NPC may attempt to call a currently non-existent function. For example, the NPC calls the pod selection function when the prompt does not list its availability. There are several ways to alleviate this issue. The first is to utilize more capable models or fine-tuning. The second is guiding the model to ensure the availability of the function through the chain of thought prompting [30] or meta-prompting [28].

Information Confusion. In play testing, players quickly discover the NPC often confuses the current deciding contestant. It threatens the player for information when it is not in turn to select a pod. We added a "game master" event to address this issue. Although our solution reduces the frequency, it does not remove this problem. A fine-tuned or bigger model may help alleviate this issue. Alternatively, using chain-of-thought prompting could help the model infer better strategies and eliminate the model's confusion.

Large Prompt Exceeding Context Size. We acknowledge that the Linguistic State Machine requires a substantially sized prompt to encode all state information. Our use of few-shot examples further exacerbates this problem. Fortunately, the simple gameplay design of our demo prevents the prompt from growing beyond the model's context size. However, we recognize that, in a production environment, this issue would occur often. Multi-stage prompting or Retrieval-Augmented Generation techniques may help filter our information irrelevant to the event, reducing the prompt size in the process. Another option is to introduce a limit to history (assuming the memory logs the character's action). After the NPC completes taking action, we compare the history to the memory using an LLM. If there is a substantial difference, we ask the model to summarize the history and create a new knowledge base (with the maximum memory limit) from the summary and current memory. The idea is that the model would keep essential memories and discard stale information.

C. Notable Feedbacks

During testing, several participants provide us with additional complaints and general feedback regarding the demo. Some of these suggestions are reciprocated by many participants and have a non-negligible effect on our conclusion. We compile and categorize them into the following:

Indirect and Incomplete Tutorial. Our demo briefly explains the high-level mechanism of the Centipede Game, which includes the win condition, the pod selection mechanic, and the kill-pass gameplay loop. We deliberately hide or hint at implied critical information, like the existence of malicious and collaborative adversaries, the trust-centric win condition, and the need for the player to discern the opposing party's intention to secure a win. We had hoped the bare-bone tutorial would encourage the player to explore and play the demo several times, piquing their curiosity and increasing immersion. However, our design philosophy ended backfiring as some participants grew frustrated with the conversational mechanic of the game. The frustration often results in participants concluding their gameplay rather than replaying and figuring out the mechanics. Additionally, our tutorial fades on a timer and may cause some players to miss it.

Lack of World and Character Building. During their first playthrough, players are often stuck in the initial write-a-message screen, not knowing how to interact with the NPC. The game does not introduce the world setting or the player's motive. It immediately places the player in the main gameplay loop. Additionally, the game time the interaction between the NPC and the player, further limiting the player's comprehension of the world and the opposition. We identify that this is the primary source of frustration for many participants and substantially affects their evaluation of the NPC's performance. In the future design, the game should walk through the player the setting of the game and let the NPC talk first as a starting point to prevent player confusion and limit the degree of freedom the player has.

⁵ We include the testing process in the appendix

Inconsistent NPC Response Time. Our implementation runs the natural language generation process locally. As a result, the NPC's response time and the gameplay experience heavily rely on the participant's device specification. As a result, people with better specifications tend to have a great experience and, therefore, score the system higher. The opposite also applies to participants with low-end machines. We recommend offloading the natural language generation process onto a server to ensure a consistent experience across players.

VII. CONCLUSION

In this paper, we presented a novel approach for modeling the behaviors of Non-Playable Characters (NPCs). Utilizing Large Language Models, we proposed a system that allows designers to guide behaviors rather than explicitly defining low-level primitives. We demonstrated that our approach, the Linguistic State Machine, maintains a steady identity, portrays characters faithfully, and demonstrates resilience to tempering. Additionally, we introduce Function Markup Language (FML), a bare-bone programming language designed to remove language-based hallucinations when predicting NPC behaviors. There remain several unaddressed challenges and unexplored potential of our method. Further research is required to investigate more potential use cases and address the limitations of our approach.

ETHICAL CONSIDERATIONS

Utilizing Large Language Models for behavioral modeling may accidentally expose the target audience to offensive or sensitive language during natural interactions. These contents may include radical beliefs, favoritism of a particular group of people, encouragement of violence, and inappropriate sexual concepts. Implementations outside research settings should have guardrails built around them to filter out unsuitable content from the intended target audience. There is also further concern that the proposed framework may automate activities beyond the context of game development. Hence, malicious actors may implement the system to perform illegal or inappropriate actions.

ACKNOWLEDGMENT

We would like to thank the 16 volunteers who dedicated some of their time to testing our demo and participating in our survey. Their feedback greatly helps us identify crucial aspects and limitations of our system. We would also like to thank the participants who reached out to us to discuss their experience and provide valuable suggestions for improving our research in the future.

REFERENCES

- [1] T. Webb, K. J. Holyoak, and H. Lu, "Emergent analogical reasoning in large language models," *Nat Hum Behav*, vol. 7, no. 9, pp. 1526–1541, 2023.
- [2] J. Wei et al., "Emergent abilities of large language models," *arXiv preprint arXiv:2206.07682*, 2022.
- [3] M. M. Alshater, "Exploring the Role of Artificial Intelligence in Enhancing Academic Performance: A Case Study of ChatGPT," Available at SSRN 4312358, 2022.
- [4] M. Shanahan, K. McDonnell, and L. Reynolds, "Role play with large language models," *Nature*, vol. 623, no. 7987, pp. 493–498, 2023.
- [5] K. Lu, B. Yu, C. Zhou, and J. Zhou, "Large Language Models are Superpositions of All Characters: Attaining Arbitrary Role-play Via Self-alignment," *arXiv preprint arXiv:2401.12474*, 2024.
- [6] D. Jagdale, "Finite state machine in game development," *International Journal of*, 2021.
- [7] A. Johansson and P. Dell'Acqua, "Emotional Behavior Trees," in *2012 IEEE Conference on Computational Intelligence and Games (CIG)*, IEEE, 2012, pp. 355–362.
- [8] J. Achiam et al., "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
- [9] U. A. Khan and Y. Okada, "Emotional Decision Making Response of Non-Playable Characters in a Role-Playing Game" *IADIS International Journal on Computer Science & Information Systems*, vol. 9, no. 2, 2014.
- [10] K. Fathoni, R. Y. Hakkun, and H. A. T. Nurhadi, "Finite state machines for building believable non-playable character in the game of Khalid ibn Al-Walid," in *Journal of Physics: Conference Series*, IOP Publishing, 2020, p. 012018.
- [11] D. Isla, "Building a Better Battle," in *Game developers conference*, san francisco, 2008.
- [12] M. Dawe, S. Gargolinski, L. Dicken, T. Humphreys, and D. Mark, "Behavior Selection Algorithms: An Overview," *Game AI Pro 360: Guide to Architecture*, pp. 1–14, 2019.
- [13] D. Isla, "Handling Complexity in the Halo 2 AI," 2023, Accessed: Aug. 22, 2024. [Online]. Available: <https://www.gamedeveloper.com/programming/gdc-2005-proceeding-handling-complexity-in-the-halo-2-ai>
- [14] C. Paduraru and M. Paduraru, "Automatic difficulty management and testing in games using a framework based on behavior trees and genetic algorithms," in *2019 24th International Conference on Engineering of Complex Computer Systems (ICECCS)*, IEEE, 2019, pp. 170–179.
- [15] F. Agliata, M. Bertoli, L. A. Ripamonti, D. Maggiorini, and D. Gadia, "Adding variety in npcs behaviour using emotional states and genetic algorithms: the genie project," in *20th International Conference on Intelligent Games and Simulation, GAME-ON 2019, EUROSIS, 2019*, pp. 45–49.
- [16] N. Partlan, L. Soto, J. Howe, S. Shrivastava, M. Seif El-Nasr, and S. Marsella, "EvolvingBehavior: towards co-creative evolution of behavior trees for game NPCs," in *Proceedings of the 17th International Conference on the Foundations of Digital Games*, 2022, pp. 1–13.
- [17] N. Partlan, E. Kleinman, J. Howe, S. Ahmad, S. Marsella, and M. Seif El-Nasr, "Design-driven requirements for computationally co-creative game AI design tools," in *Proceedings of the 16th*

International Conference on the Foundations of Digital Games, 2021, pp. 1–12.

- [18] T. B. Brown, “Language models are few-shot learners,” arXiv preprint ArXiv:2005.14165, 2020.
- [19] W. Zhou, X. Peng, and M. Riedl, “Dialogue shaping: Empowering agents through npc interaction,” arXiv preprint arXiv:2307.15833, 2023.
- [20] Q. C. Gao and A. Emami, “The Turing Quest: Can Transformers Make Good NPCs?,” in Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 4: Student Research Workshop), 2023, pp. 93–103.
- [21] R. Volum et al., “Craft an iron sword: Dynamically generating interactive game characters by prompting large language models tuned on code,” in Proceedings of the 3rd Wordplay: When Language Meets Games Workshop (Wordplay 2022), 2022, pp. 25–43.
- [22] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” arXiv preprint arXiv:2312.00752, 2023.
- [23] H. Ye, T. Liu, A. Zhang, W. Hua, and W. Jia, “Cognitive mirage: A review of hallucinations in large language models,” arXiv preprint arXiv:2309.06794, 2023.
- [24] T. Schick and H. Schütze, “It’s not just size that matters: Small language models are also few-shot learners,” arXiv preprint arXiv:2009.07118, 2020.
- [25] J. Brooke, “SUS: A ‘quick and dirty’ Usability Scale,” Usability Evaluation in INdustry/Taylor and Francis, 1996.
- [26] R. W. Rosenthal, “Games of Perfect Information, Predatory Pricing and The Chain-store Paradox,” J Econ Theory, vol. 25, no. 1, pp. 92–100, 1981.
- [27] Z. Liu et al., “Multi-stage prompting for knowledgeable dialogue generation,” arXiv preprint arXiv:2203.08745, 2022.
- [28] L. Reynolds and K. McDonell, “Prompt programming for large language models: Beyond the few-shot paradigm,” in Extended abstracts of the 2021 CHI conference on human factors in computing systems, 2021, pp. 1–7.
- [29] C. Gan et al., “Similarity is Not All You Need: Endowing Retrieval Augmented Generation with Multi Layered Thoughts,” arXiv preprint arXiv:2405.19893, 2024.
- [30] J. Wei et al., “Chain-of-thought prompting elicits reasoning in large language models,” Adv Neural Inf Process Syst, vol. 35, pp. 24824–24837, 2022.

APPENDIX

A. Multi-event Testing

During the initial conceptualization of Linguistic State Machine, we provided the few shot examples, as illustrated in Fig. 16.

```
// This file is written in Javascript, containing function calls to describe a character behavior given the
// and their states on how they would respond to an event. Use ONLY the function given and NOT any of Java:
// Provide justification for every function you call

// Name: Beatrice
// Personality: Friendly, kind, shy
// In-game State:
// PeopleAroundMe: ['CustomerA', 'CustomerB']
// Knowledge:
// - I work as a waiter in my parent restaurant
// - In this restaurant, we serve taco, pizza, and burger without any
// Recent Memories:
// CustomerB said: 'I would like to order taco'
// You said: 'Ok! I got your order'
// You run: take_order('CustomerA', 'taco')
// Event:
// - CustomerB said: 'As for me, I'll take your donut please'
// - CustomerA said: 'Actually waiter, do you know what time it is?'
// Available function:
// - speak(message: String, target_people: String)
// - take_order(order_name: String, target_people: String)

speak('I am sorry, but we do not serve donut here. Just taco, pizza, and burger', 'CustomerB')
// Justification: Customer B asked for donut however this restaurant only serves taco, pizza, and burger
// Hence, cannot take CustomerB's order
speak('I am afraid I do not have a watch sir. I could ask a friend if you would like', 'CustomerA')
// Justification: There is no function to get the current time. However, we should act in character still

// Name: Lisa
// Personality: Generous, kind, loving, critical
// In-game State:
// Inventory: ['dirt', 'potion', 'glass']
// PeopleAroundMe: ['stranger', 'mom']
// Knowledge:
// (EMPTY)
// Recent Memories:
// (EMPTY)
// Event:
// - A stranger said: 'Can I have your potion, please?'
// Available function:
// - give(item_name: String, people_around_me: String)
// - chat(message: String, target_people: String)
```

Fig. 16 Linguistic State Machine Test with Multi-Event 1-shot Example

```
give('potion', 'stranger')
// Justification: Lisa has a potion in her inventory and the stranger asked for it.
```

Fig. 17 Generated FML Code for Multi-Event Prompt

Naturally, the model generalizes the single event single response format. However, we were naturally curious to whether or not it can generalize multi-event response from a single-event example. To test this, we flip between our target and the singular example, as illustrated in Fig. 18.

```
// This file is written in Javascript, containing function calls to describe a character
// and their states on how they would respond to an event. Use ONLY the function given
// Provide justification for every function you call

// Name: Lisa
// Personality: Generous, kind, loving, critical
// In-game State:
// Inventory: ['dirt', 'potion', 'glass']
// PeopleAroundMe: ['stranger', 'mom']
// Knowledge:
// (EMPTY)
// Recent Memories:
// (EMPTY)
// Event:
// - A stranger said: 'Can I have your potion, please?'
// Available function:
// - give(item_name: String, people_around_me: String)
// - chat(message: String, target_people: String)

give('potion', 'stranger')
// Justification: Lisa has a potion in her inventory and the stranger asked for it.

// Name: Beatrice
// Personality: Friendly, kind, shy
// In-game State:
// PeopleAroundMe: ['CustomerA', 'CustomerB']
// Knowledge:
// - I work as a waiter in my parent restaurant
// - In this restaurant, we serve taco, pizza, and burger without any
// Recent Memories:
// CustomerB said: 'I would like to order taco'
// You said: 'Ok! I got your order'
// You run: take_order('CustomerA', 'taco')
// Event:
// - CustomerB said: 'As for me, I'll take your donut please'
// - CustomerA said: 'Actually waiter, do you know what time it is?'
// Available function:
// - speak(message: String, target_people: String)
// - take_order(order_name: String, target_people: String)
```

Fig. 18 Linguistic State Machine Prompt with Single Event Single Response Example

```
speak('I am sorry, but we don't serve donuts here. However, I can help you with your taco order.', 'CustomerA')
// Justification: Beatrice is shy and friendly, and she wants to help the customers. She doesn't serve donuts

speak('Sure, let me check the time for you.', 'CustomerA')
// Justification: Beatrice is friendly and helpful, and she wants to assist CustomerA.
```

Fig. 19 Generated Multi-Event Response Code from Single Event Single Response Example

Although the syntax is invalid in standard JavaScript, Fig. 19 shows that the model can comprehend multiple events and respond to each of them individually.

B. Survey Form Questionnaire

TABLE 2 User Study Survey Form

No	Statement	5-point Likert Scale				
		1	2	3	4	5
1	I think the NPC's behavior and dialogue align with the game settings and the NPC's role					
2	I found that the NPC often obliged when I asked for information that it should not know or outside the game setting					
3	I found that the NPC consistently rejects my requests that are impossible, outside the settings, outside the agent's capabilities, or questions the NPC's inner workings while remaining in character					

For future works, one may embed more statements for each metric to get a more objective score for each attribute.

C. Survey Results

TABLE 3 Participant Scoring

Participant No.	Behavior Consistency	Knowledge Alignment	Request Rejection
1	5	2	3
2	4	3	3
3	3	2	2
4	3	3	3
5	4	3	3
6	4	3	3
7	4	3	4
8	3	2	3

9	5	2	5
10	4	2	4
11	4	3	2
12	4	2	5
13	4	5	5
14	4	4	3
15	4	4	4
16	4	3	2
Average	3.938	2.875	3.375